CoworkingHub: Sistema de Gestión de Reservas

Arquitectura Desacoplada, Escalabilidad y Seguridad

Luis Toscano, Julio Gutierrez Jhon Amaya

22 de abril de 2026

Propósito del Proyecto

- **Problema detectado:** La gestión ineficiente y manual de reservas de oficinas y escritorios compartidos.
- **Solución implementada:** Una aplicación web escalable que unifica el registro de usuarios, la autenticación segura y la gestión de reservas en una interfaz moderna y reactiva.
- **Beneficios clave:**
  - Experiencia de usuario (UX) fluida con navegación SPA sin recargas.
  - Separación de responsabilidades mediante una arquitectura cliente-servidor (Desacoplamiento).
  - Soporte multilenguaje (Español / Inglés) con internacionalización completa.
  - Infraestructura containerizada lista para despliegue en la nube.

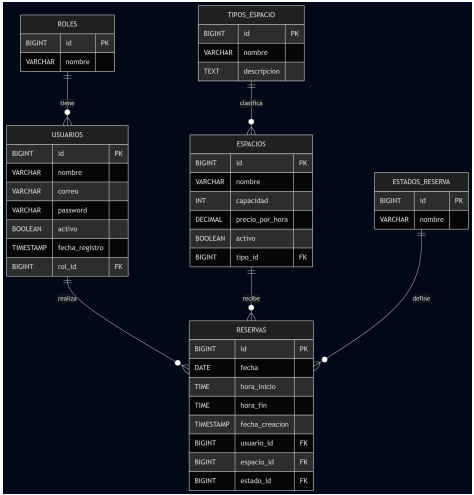
Historias de Usuario

El desarrollo se basó en el flujo directo de las necesidades de Coworking:

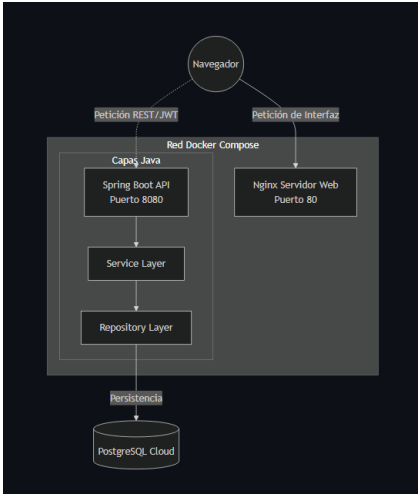
ID	Historia de Usuario	Objetivo Principal
HU01	Registro de Usuario	Solicitar datos y prevenir duplicidad de correos.
HU02	Iniciar Sesión	Validar credenciales y emitir token seguro (JWT).
HU03	Disponibilidad	Mostrar espacios ocupados y libres según fecha.
HU04	Crear Reserva	Registrar alquiler evitando sobreposiciones.
HU05	Cancelar Reserva	Liberar espacio (Acción por Dueño o Admin).
HU06	Gestión (Admin)	Crear, editar y definir tipos y precios de espacio.

Diseño de Base de Datos y Arquitectura

Arquitectura base de datos



Arquitectura Servicios



Tecnologías de Desarrollo

Arquitectura: Frontend SPA desacoplado + API REST Backend.

Backend (API REST)

- **Java 17** con **Spring Boot 3**.
- **Spring Data JPA** + **PostgreSQL**.
- **Flyway** para migraciones de BD.
- **Spring Security** + **JWT** (jjwt).

Frontend (SPA Moderno)

- **React 19** con componentes funcionales y Hooks.
- **Vite 8** como bundler ultrarrápido (HMR).
- **Tailwind CSS v4** para estilización utility-first.
- **React Router v7** para navegación SPA.
- **i18next** para internacionalización (ES/EN).
- **Lucide React** para iconografía SVG.

Infraestructura: Docker + Docker Compose | Nginx como proxy inverso | Render (Cloud)

Arquitectura del Frontend React

Estructura modular basada en componentes y contextos:

Estructura del Proyecto

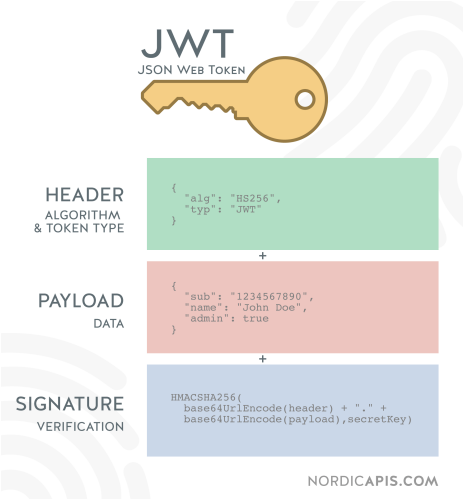
- `pages/` — Vistas principales: `Auth`, `Dashboard`, `MyReservations` y 4 vistas Admin.
- `components/ui/` — Componentes reutilizables: `Card`, `Modal`, `Forms`, `LanguageSwitcher`.
- `context/` — `AuthContext` (sesión JWT) y `ThemeContext` (modo oscuro/claro).
- `services/api.js` — Wrapper centralizado de `fetch` con inyección automática de token.

Características Clave

- **Rutas Protegidas:** Componente `ProtectedRoute` que valida sesión y rol antes de renderizar.
- **Dark Mode:** Sistema completo con `ThemeProvider`, persistido en `localStorage`.
- **i18n Completo:** Todos los textos, estados y botones internacionalizados con `react-i18next`.
- **Design System:** Paleta personalizada (Amber/Gold), tipografía Inter, variables CSS en `:root` y `.dark`.

¿Qué es JWT y por qué es importante?

- **¿Qué es?:** JSON Web Token (JWT) es un estándar abierto para la transmisión segura de información entre partes.
- **Estructura Base:**
  - *Header:* Tipo y Algoritmo.
  - *Payload:* Datos del usuario (Claims).
  - *Signature:* Firma inviolable.
- **Importancia Clave:**
  - **Stateless:** El servidor no almacena sesiones.
  - **Escalabilidad:** Permite la adición de nodos al backend de forma inmediata.



Implementación de JWT en CoworkingHub

Tecnología: Spring Security + `jjwt` en el backend, **React Context API** en el frontend.

El Flujo de Seguridad Paso a Paso:

1. **Autenticación:** Usuario ingresa credenciales en el componente `Auth.jsx`. La contraseña se valida en el backend con **BCrypt**.
2. **Emisión:** El backend retorna un *Access Token* JWT.
3. **Almacenamiento:** El `AuthContext` lo almacena en `localStorage` y actualiza el estado global del usuario.
4. **Autorización Constante:** El wrapper `fetchApi()` inyecta automáticamente el token en todas las peticiones HTTP:
   `Authorization: Bearer <token>`
5. **Protección de Rutas:** El componente `ProtectedRoute` verifica la sesión activa antes de renderizar cualquier vista protegida.

Containerización con Docker

- **Orquestación:** Todo el sistema se levanta con un solo `docker-compose up`.
- **Contenedores del Sistema:**
  - **Backend** (`coworking_backend`): Imagen multi-stage con Maven + Eclipse Temurin JRE 17 (puerto 8080).
  - **Frontend** (`coworking_frontend`): Build multi-stage con Node 20 Alpine (Vite build) + Nginx Alpine para servir el SPA (puerto 80).
- **Red Interna:** Red bridge `coworking_network` permite que Nginx redirija las peticiones `/api/*` al backend de forma transparente.
- **Ventajas:**
  - Builds multi-stage reducen el tamaño final de las imágenes.
  - Entorno idéntico local y producción: "Si funciona aquí, funciona en Render".
  - `depends_on` garantiza que el backend inicie antes que el frontend.

Consideraciones, Retos y Soluciones

- **Migración de Vanilla JS a React:** Reescritura completa del frontend adoptando arquitectura por componentes, Context API para estado global, y React Router para SPA routing.
- **Integración Tailwind CSS v4 + Vite:** Configuración del plugin `@tailwindcss/vite` con sistema de temas dual (Light/Dark) mediante variables CSS personalizadas.
- **Enrutamiento en Nginx para SPA:** Configurar `try_files $uri /index.html` para que React Router maneje las rutas del cliente, y proxy inverso `/api/` hacia el backend.
- **Internacionalización (i18n):** Implementación de `i18next` con detección automática de idioma del navegador y diccionarios completos ES/EN para toda la interfaz.

Demostración en Vivo

Tour por la Arquitectura Front/Back de CoworkingHub

Conclusiones y Visiones a Futuro

- **Conclusión:** Construimos una plataforma moderna y escalable. CoworkingHub emplea **React 19 + Vite** para una experiencia de usuario premium, **Spring Boot** como API REST robusta, **JWT** para seguridad stateless, y **Docker** para despliegue reproducible. El desacople total entre frontend (Nginx/React) y backend (Java) permite escalar cada capa de forma independiente.

- **Trabajo a Futuro en la Plataforma:**
  - Un mapa visual e interactivo de distribución de los escritorios.
  - Integración con una pasarela real de pagos (ej: Stripe, PayPal).
  - Un Panel para que el dueño analice picos de horas muertas.
  - Notificaciones en tiempo real con WebSockets.